IPSSI

2e année Bachelor Informatique

Mathématiques appliquées à l'Intelligence Artificielle

Corrections détaillées

Jour 1 — Données et représentation mathématique

Auteur : Mohamed EL AFRIT

www.mohamedelafrit.com

Année académique : 2025–2026

© 2025 Mohamed EL AFRIT — IPSSI

Ce contenu est distribué sous licence

Creative Commons BY-NC-SA 4.0

https://creativecommons.org/licenses/by-nc-sa/4.0/deed.fr

Rappel — Dataset fil rouge

#	Exp	Lang	Études	Entrep.	Remote	Salaire
0	0	2	3	150	80	28.0
1	1	1	2	50	100	26.5
2	1	3	4	800	40	35.0
3	2	2	3	200	60	32.0
4	2	4	4	3000	20	38.0
5	0	1	1	30	50	25.0

Exercice 1

★★☆ Fondamental

Rappel : écrire des vecteurs de features et vérifier les dimensions.

a) Vecteur $\mathbf{x}_0$ du développeur #0.
Le développeur #0 a les features : exp=0, lang=2, études=3, entrep.=150, remote=80.

$$\mathbf{x}_0 = \begin{pmatrix} 0 \\ 2 \\ 3 \\ 150 \\ 80 \end{pmatrix} \in \mathbb{R}^5 \qquad \Longrightarrow \qquad p = 5$$

b) Vecteur $\mathbf{x}_4$ et sa transposée.

$$\mathbf{x}_4 = \begin{pmatrix} 2 \\ 4 \\ 4 \\ 3000 \\ 20 \end{pmatrix} \qquad \mathbf{x}_4^T = \begin{pmatrix} 2 & 4 & 4 & 3000 & 20 \end{pmatrix}$$

c) Nouveau développeur (6 ans, 4 langages, Bac+5, 1200 employés, 50% remote) :

$$\mathbf{x} = \begin{pmatrix} 6 \\ 4 \\ 4 \\ 1200 \\ 50 \end{pmatrix} \in \mathbb{R}^5$$

d) Addition de vecteurs. $\mathbf{x}_0 + \mathbf{x}_4$: **oui**, car les deux sont dans $\mathbb{R}^5$ (même dimension).

$$\mathbf{x}_0 + \mathbf{x}_4 = \begin{pmatrix} 0+2 \\ 2+4 \\ 3+4 \\ 150+3000 \\ 80+20 \end{pmatrix} = \begin{pmatrix} 2 \\ 6 \\ 7 \\ 3150 \\ 100 \end{pmatrix}$$

 CC BY-NC-SA 4.0

$\mathbf{x}_0 + (1,\ 2,\ 3)^T$: **non**, car $\mathbf{x}_0 \in \mathbb{R}^5$ et $(1, 2, 3)^T \in \mathbb{R}^3$. L'addition n'est définie que pour des vecteurs de **même dimension**.

Erreurs fréquentes

- Confondre la **dimension** du vecteur ($p = 5$) avec le **nombre d'observations** ($n = 30$).
- Oublier la **transposée** : un vecteur colonne $\mathbf{x}$ et sa transposée $\mathbf{x}^T$ n'ont pas le même shape en NumPy.

Exercice 2

★☆☆ **Fondamental**

Rappel : $\langle \mathbf{x}, \mathbf{w} \rangle = \sum_{i=1}^{p} x_i w_i$.

---

a) Calcul de $\langle \mathbf{x}, \mathbf{w} \rangle$:

$$\begin{aligned}\langle \mathbf{x}, \mathbf{w} \rangle &= 3 \times 0{,}5 + 5 \times 0{,}3 + 2 \times 0{,}8 \\ &= 1{,}5 + 1{,}5 + 1{,}6 \\ &= \boxed{4{,}6}\end{aligned}$$

b) Prédiction pour le développeur #0 avec $\mathbf{w} = (2{,}0;\ 1{,}5;\ 3{,}0;\ 0{,}005;\ 0{,}02)^T$ et $b = 15$:

$$\begin{aligned}\langle \mathbf{x}_0, \mathbf{w} \rangle &= 0 \times 2{,}0 + 2 \times 1{,}5 + 3 \times 3{,}0 + 150 \times 0{,}005 + 80 \times 0{,}02 \\ &= 0 + 3{,}0 + 9{,}0 + 0{,}75 + 1{,}6 = 14{,}35\end{aligned}$$

Prédiction : $\hat{y}_0 = 14{,}35 + 15 = \boxed{29{,}35}$ k€. Salaire réel : $y_0 = 28{,}0$ k€. Erreur : $|29{,}35 - 28{,}0| = 1{,}35$ k€.

c) Piège — dimensions incompatibles.

Non, on ne peut pas calculer ce produit scalaire. Le premier vecteur est dans $\mathbb{R}^5$ et le second dans $\mathbb{R}^3$. Le produit scalaire exige que les deux vecteurs aient la **même dimension**.

Erreurs fréquentes

- Oublier d'additionner le **biais** b après le produit scalaire.
- Erreur de calcul sur les termes avec des petits poids ($0{,}005 \times 150$ se calcule mieux comme $150/200 = 0{,}75$).
- Tenter de calculer un produit scalaire avec des dimensions différentes — en NumPy, cela provoque une `ValueError`.

Exercice 3

★☆☆ **Fondamental**

Rappel : $(\mathbf{X}\mathbf{w})_i = \mathbf{x}_i^T \mathbf{w}$ *(produit scalaire de la ligne i avec $\mathbf{w}$).*

---

a) Dimensions. $\mathbf{X} \in \mathbb{R}^{3\times 2}$, $\mathbf{w} \in \mathbb{R}^2$, donc $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \in \mathbb{R}^3$ (3 prédictions).

b) Calcul ligne par ligne :

$$\hat{y}_1 = 1 \times 2 + 3 \times 0{,}5 = 2 + 1{,}5 = \boxed{3{,}5}$$
$$\hat{y}_2 = 4 \times 2 + 2 \times 0{,}5 = 8 + 1{,}0 = \boxed{9{,}0}$$
$$\hat{y}_3 = 0 \times 2 + 5 \times 0{,}5 = 0 + 2{,}5 = \boxed{2{,}5}$$

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \begin{pmatrix} 3{,}5 \\ 9{,}0 \\ 2{,}5 \end{pmatrix}$$

c) Vérification : $\mathbf{x}_1^T = (1 \;\; 3)$, donc $\mathbf{x}_1^T \mathbf{w} = 1 \times 2 + 3 \times 0{,}5 = 3{,}5 = \hat{y}_1$. ✓

À retenir

La multiplication $\mathbf{X}\mathbf{w}$ revient à n produits scalaires en parallèle. Chaque ligne de $\mathbf{X}$ fait un produit scalaire avec $\mathbf{w}$.

Exercice 4

★★☆ **Fondamental**

Rappel : MSE $= \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$.

a) Résidus $e_i = y_i - \hat{y}_i$ **:**

i	y_i	$\hat{y}_i$	e_i	e_i^2
1	28.0	31.3	−3,3	10,89
2	35.0	33.3	+1,7	2,89
3	38.0	35.4	+2,6	6,76
4	44.0	41.6	+2,4	5,76
5	52.0	49.9	+2,1	4,41
			Somme	**30,71**

b-c) Calcul du MSE :

$$\text{MSE} = \frac{1}{5} \times 30{,}71 = \boxed{6{,}14\ (\text{k€})^2}$$

d) Erreur typique : $\sqrt{\text{MSE}} = \sqrt{6{,}14} \approx \boxed{2{,}48\ \text{k€}}$. Le modèle se trompe en moyenne d'environ 2 500 € — c'est raisonnable.

Erreurs fréquentes

- Oublier de mettre au **carré** avant de sommer (sinon les erreurs + et − s'annulent).
- Diviser par $n - 1$ au lieu de n (c'est la variance *non biaisée*, pas le MSE).
- Le MSE est en **(k€)**^{2}, pas en k€. C'est $\sqrt{\text{MSE}}$ qui est en k€.

Exercice 5

★★☆ Intermédiaire

Modèle : $\hat{y} = 25 + 3 \times \mathit{exp} + 1{,}5 \times \mathit{lang}$.

a) Identification des paramètres :

— $w_0 = 25$ (biais) : salaire de base prédit quand expérience $= 0$ et langages $= 0$.

— $w_1 = 3$: chaque année d'expérience supplémentaire augmente le salaire prédit de **3 k€**.

— $w_2 = 1{,}5$: chaque langage supplémentaire augmente le salaire prédit de **1,5 k€**.

b) $\hat{y} = 25 + 3 \times 0 + 1{,}5 \times 1 = \boxed{26{,}5}$ k€. C'est réaliste pour un débutant avec un seul langage en France.

c) +1 an d'expérience ⇒ +3 k€. +1 langage ⇒ +1,5 k€. L'expérience a **deux fois plus d'impact** que le nombre de langages.

d) $\hat{y} = 25 + 3 \times 5 + 1{,}5 \times 4 = 25 + 15 + 6 = \boxed{46}$ k€. Le développeur #9 a un vrai salaire de 44 k€ — erreur de 2 k€, le modèle est assez bon ici.

À retenir

Les poids d'un modèle linéaire s'interprètent comme l'**impact marginal** de chaque feature sur la prédiction, *toutes choses égales par ailleurs*.

Exercice 6

★★☆ Intermédiaire

Modèle A : $\hat{y} = 2x + 32$. *Modèle B :* $\hat{y} = 3x + 25$.

a-b) Prédictions :

#	exp	y_i	$\hat{y}_i^A = 2x{+}32$	$\hat{y}_i^B = 3x{+}25$	$(e_i^A)^2$	$(e_i^B)^2$
0	0	28.0	32.0	25.0	16.00	9.00
1	1	26.5	34.0	28.0	56.25	2.25
2	1	35.0	34.0	28.0	1.00	49.00
3	2	32.0	36.0	31.0	16.00	1.00
4	2	38.0	36.0	31.0	4.00	49.00
5	0	25.0	32.0	25.0	49.00	0.00

c) MSE :

$$\mathrm{MSE}_A = \frac{16{,}00 + 56{,}25 + 1{,}00 + 16{,}00 + 4{,}00 + 49{,}00}{6} = \frac{142{,}25}{6} = \boxed{23{,}71}$$

$$\mathrm{MSE}_B = \frac{9{,}00 + 2{,}25 + 49{,}00 + 1{,}00 + 49{,}00 + 0{,}00}{6} = \frac{110{,}25}{6} = \boxed{18{,}38}$$

d) Le modèle B est meilleur (MSE plus faible). Cependant, ni A ni B n'est optimal : la droite des moindres carrés ($\hat{y} = 2{,}07x + 31{,}27$) a un MSE de 20,46 sur les 30 observations complètes. Sur un si petit échantillon (6 observations), le classement peut changer.

⚠ Erreurs fréquentes

- Comparer les MSE de modèles évalués sur des échantillons **différents** — il faut toujours utiliser les mêmes données.
- Conclure qu'un modèle est "le meilleur" alors qu'il n'est meilleur que sur ce sous-ensemble.

Exercice 7

★★☆ Intermédiaire

Construire $\mathbf{X}$ *et* $\mathbf{y}$ *à partir de profils textuels.*

---

a) Matrice de features $\mathbf{X} \in \mathbb{R}^{4\times 5}$:
Encodage : Bac= 1, Bac+3= 3, Bac+5= 4.

$$\mathbf{X} = \begin{pmatrix} 3 & 2 & 3 & 150 & 80 \\ 7 & 5 & 4 & 800 & 50 \\ 0 & 1 & 1 & 20 & 100 \\ 12 & 6 & 4 & 3000 & 30 \end{pmatrix} \in \mathbb{R}^{4\times 5}$$

b) Vecteur cible : $\mathbf{y} = \begin{pmatrix} 34 & 50 & 24 & 58 \end{pmatrix}^T \in \mathbb{R}^4$.

c) $x_{2,4} = 100$. C'est le pourcentage de télétravail de Clara (2e ligne, 4e feature en partant de 0 — ou 3e ligne, 5e colonne en indexation à partir de 1).

⚠ Attention à l'indexation

L'indexation mathématique commence à **1** ($x_{1,1}$) tandis que Python/NumPy commence à **0** (`X[0,0]`). Ici Bob est la ligne 2 en math (ou `X[1]` en Python).

d) Prédiction pour Bob : $\mathbf{x}_{\text{Bob}} = (7,\ 5,\ 4,\ 800,\ 50)^T$.

$$\begin{aligned} \mathbf{w}^T\mathbf{x}_{\text{Bob}} &= 7 \times 2 + 5 \times 1{,}5 + 4 \times 3 + 800 \times 0{,}005 + 50 \times 0{,}02 \\ &= 14 + 7{,}5 + 12 + 4 + 1 = 38{,}5 \end{aligned}$$

$\hat{y}_{\text{Bob}} = 38{,}5 + 15 = \boxed{53{,}5}$ k€. Vrai salaire : 50 k€ — erreur de 3,5 k€.

Exercice 8

★★☆ Intermédiaire

Créer des vecteurs, calculer norme et produit scalaire.

CC BY-NC-SA 4.0

» Correction complète

```python
import numpy as np

# 1. Vecteur du dev #3 : exp=2, lang=2, etudes=3, entrep=200, remote=60
x3 = np.array([2, 2, 3, 200, 60])

# 2. Dimension
print(f"Dimension de x3 : {x3.shape}")  # (5,)

# 3. 3e composante (index 2 en Python)
niveau = x3[2]
print(f"Niveau d'etudes : {niveau}")  # 3

# 4. Norme euclidienne
norme = np.sqrt(np.sum(x3 ** 2))
print(f"Norme de x3 : {norme:.2f}")  # 209.11

# 5. Vecteur du dev #5 : exp=0, lang=1, etudes=1, entrep=30, remote=50
x5 = np.array([0, 1, 1, 30, 50])

# 6. Produit scalaire avec @
ps = x3 @ x5
print(f"Produit scalaire x3 . x5 = {ps}")  # 9185

# 7. Distance euclidienne
distance = np.sqrt(np.sum((x3 - x5) ** 2))
print(f"Distance : {distance:.2f}")  # 170.27
```

Sortie attendue :

```
Dimension de x3 : (5,)
Niveau d'etudes : 3
Norme de x3 : 209.11
Produit scalaire x3 . x5 = 9185
Distance : 170.27
```

Variante

La norme peut aussi se calculer avec `np.linalg.norm(x3)`. Le produit scalaire avec `np.dot(x3, x5)`. L'opérateur `@` est le plus lisible et le plus courant en pratique.

Erreurs fréquentes en Python

- `x3[3]` donne la 4e composante (index 3), pas la 3e — l'indexation commence à 0.
- La norme de **x** est dominée par les grandes valeurs (ici 200 et 60). C'est pourquoi on **normalise** souvent les features.

Exercice 9

★★☆ Intermédiaire

Charger les données dans une matrice, slicing lignes/colonnes.

» Correction complète

```python
import numpy as np

# 1. Matrice des 6 premiers developpeurs (5 features)
X = np.array([
    [0, 2, 3, 150, 80],      # dev #0
    [1, 1, 2, 50, 100],      # dev #1
    [1, 3, 4, 800, 40],      # dev #2
    [2, 2, 3, 200, 60],      # dev #3
    [2, 4, 4, 3000, 20],     # dev #4
    [0, 1, 1, 30, 50],       # dev #5
])

# 2. Verifier les dimensions
print(f"Shape de X : {X.shape}")      # (6, 5)

# 3. Colonne "experience" = colonne d'index 0
experience = X[:, 0]
print(f"Experiences : {experience}")  # [0 1 1 2 2 0]

# 4. Ligne du developpeur #4 = index 4
dev4 = X[4, :]
print(f"Developpeur #4 : {dev4}")     # [2 4 4 3000 20]

# 5. Vecteur des salaires
y = np.array([28.0, 26.5, 35.0, 32.0, 38.0, 25.0])

# 6. Transposee
print(f"Shape de X.T : {X.T.shape}") # (5, 6)

# 7. Element X[2, 3] = dev #2, feature #3 (taille entreprise)
print(f"X[2,3] = {X[2, 3]}")          # 800
# C'est la taille de l'entreprise du developpeur #2
```

Sortie attendue :

```
Shape de X : (6, 5)
Experiences : [0 1 1 2 2 0]
Developpeur #4 : [   2    4    4 3000   20]
Shape de X.T : (5, 6)
X[2,3] = 800
```

⚠ Erreurs fréquentes en Python

- Confondre `X[2]` (toute la ligne 2) et `X[:, 2]` (toute la colonne 2).
- Oublier le : dans le slicing : `X[4]` et `X[4, :]` sont équivalents, mais `X[:, 0]` est la seule façon d'extraire une colonne.

Exercice 10

★★☆ Intermédiaire

Calculer la droite des moindres carrés, le MSE, et tracer le graphique.

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

experience = np.array([0,1,1,2,2,0,3,1,4,5,5,6,6,7,7,4,
                        8,5,9,10,10,11,12,9,13,11,15,17,20,18])
salaire = np.array([28.0,26.5,35.0,32.0,38.0,25.0,34.5,42.0,
    38.5,44.0,41.0,48.0,43.5,50.0,39.0,47.0,51.0,40.0,
    52.0,55.0,48.5,58.0,60.0,42.0,62.0,54.0,65.0,68.0,
    72.0,58.0])

# 1. Moyennes
x_mean = experience.mean()          # 7.40
y_mean = salaire.mean()             # 46.58

# 2. Pente w (formule des moindres carres)
numerateur = np.sum((experience - x_mean) * (salaire - y_mean))
denominateur = np.sum((experience - x_mean) ** 2)
w = numerateur / denominateur       # 2.07
print(f"Pente w = {w:.4f}")

# 3. Biais b
b = y_mean - w * x_mean             # 31.27
print(f"Biais b = {b:.4f}")

# 4. Predictions
y_pred = w * experience + b

# 5. MSE
mse = np.mean((salaire - y_pred) ** 2)  # 20.46
print(f"MSE = {mse:.2f}")

# 6. Graphique
fig, ax = plt.subplots(figsize=(8, 5))
ax.scatter(experience, salaire, c='blue', label='Donnees')
x_line = np.linspace(0, 21, 100)
ax.plot(x_line, w * x_line + b, 'r--',
```

```
        label=f'y={w:.2f}x+{b:.2f}')
ax.set_xlabel("Experience (annees)")
ax.set_ylabel("Salaire (kEUR)")
ax.set_title("Regression lineaire from scratch")
ax.legend(); ax.grid(True, alpha=0.3)
plt.show()
```

Sortie attendue :

```
Pente w = 2.0672
Biais b = 31.2830
MSE = 20.46
```

+ un graphique montrant le nuage de points bleu et la droite rouge en pointillés.

Variante : avec scikit-learn

En une ligne : `from sklearn.linear_model import LinearRegression`. Mais l'objectif ici est de comprendre le mécanisme **avant** d'utiliser la bibliothèque.

À retenir

La formule de la pente $w_1 = \frac{\sum(x_i-\bar{x})(y_i-\bar{y})}{\sum(x_i-\bar{x})^2}$ mesure à quel point x et y varient **ensemble**, normalisé par la variance de x.

Exercice 11

★★★ Avancé

Statistiques descriptives, corrélations et visualisation.

» Correction complète

```
import numpy as np
import matplotlib.pyplot as plt

# --- Chargement du dataset ---
data = np.array([
    [ 0,2,3, 150, 80,28.0],[ 1,1,2,  50,100,26.5],
    [ 1,3,4, 800, 40,35.0],[ 2,2,3, 200, 60,32.0],
    [ 2,4,4,3000, 20,38.0],[ 0,1,1,  30, 50,25.0],
    [ 3,3,3, 100, 80,34.5],[ 1,5,4,  15,100,42.0],
    [ 4,3,3, 500, 60,38.5],[ 5,4,4,1200, 40,44.0],
    [ 5,3,3, 300, 80,41.0],[ 6,5,4,2000, 30,48.0],
    [ 6,4,3, 150, 90,43.5],[ 7,5,4, 800, 50,50.0],
    [ 7,3,2,  80, 70,39.0],[ 4,6,5, 400,100,47.0],
    [ 8,4,4,1500, 40,51.0],[ 5,2,3,4000, 10,40.0],
    [ 9,5,4, 600, 60,52.0],[10,6,4,2500, 30,55.0],
```

 CC BY-NC-SA 4.0

```python
    [10,4,3, 200, 80,48.5],[11,5,5,1000, 50,58.0],
    [12,7,4,3500, 20,60.0],[ 9,3,3,  60, 90,42.0],
    [13,6,4, 800, 70,62.0],[11,4,4,5000, 10,54.0],
    [15,8,4,1200, 50,65.0],[17,6,5,2000, 40,68.0],
    [20,7,4, 500, 60,72.0],[18,5,3, 100, 80,58.0],
])

# a) Dimensions et apercu
print(f"Dimensions : {data.shape}")
print("5 premieres lignes :")
print(data[:5])

# b) Statistiques par colonne
noms = ["experience","nb_langages","niveau_etudes",
        "taille_entreprise","remote","salaire"]
print("\n--- Statistiques descriptives ---")
for j, nom in enumerate(noms):
    col = data[:, j]
    print(f"{nom:20s} | moy={col.mean():8.2f} "
          f"std={col.std():8.2f} "
          f"min={col.min():7.1f} max={col.max():7.1f}")

# c) Correlations avec le salaire
salaire = data[:, 5]
print("\n--- Correlations avec le salaire ---")
for j in range(5):
    r = np.corrcoef(data[:, j], salaire)[0, 1]
    print(f"{noms[j]:20s} | r = {r:.3f}")

# d) 5 sous-figures
fig, axes = plt.subplots(1, 5, figsize=(18, 3.5))
for j in range(5):
    r = np.corrcoef(data[:, j], salaire)[0, 1]
    axes[j].scatter(data[:, j], salaire, s=30, alpha=0.7)
    axes[j].set_xlabel(noms[j])
    axes[j].set_ylabel("salaire" if j == 0 else "")
    axes[j].set_title(f"r = {r:.2f}")
    axes[j].grid(True, alpha=0.3)
plt.suptitle("Correlations features vs salaire")
plt.tight_layout(); plt.show()
```

Résultats clés :

Feature	r	Interprétation
experience	0,927	très forte corrélation positive
nb_langages	0,897	forte corrélation positive
niveau_etudes	0,667	corrélation modérée
taille_entreprise	0,264	faible corrélation
remote	−0,341	faible corrélation négative

L'expérience est la feature la plus corrélée au salaire, suivie du nombre de langages.

⚠ Erreurs fréquentes

- Utiliser `np.corrcoef` sans extraire l'élément `[0,1]` — la fonction renvoie une **matrice** 2×2, pas un scalaire.
- Confondre corrélation et causalité (le remote négatif ne signifie pas que le télétravail *réduit* le salaire).

Exercice 12

★★★ **Avancé**

Comparer 3 droites de régression, calculer le MSE, tracer le graphique.

» Correction complète

```python
import numpy as np
import matplotlib.pyplot as plt

# --- Donnees ---
experience = np.array([0,1,1,2,2,0,3,1,4,5,5,6,6,7,7,4,
    8,5,9,10,10,11,12,9,13,11,15,17,20,18])
salaire = np.array([28.0,26.5,35.0,32.0,38.0,25.0,34.5,
    42.0,38.5,44.0,41.0,48.0,43.5,50.0,39.0,47.0,51.0,
    40.0,52.0,55.0,48.5,58.0,60.0,42.0,62.0,54.0,65.0,
    68.0,72.0,58.0])

# --- 3 modeles ---
modeles = {
    "A": (1.5, 33),
    "B": (2.07, 31.27),
    "C": (2.8, 28),
}

# a-b) Predictions et MSE
print("Modele |  w1    |  w0    |    MSE")
print("-------|-------|-------|--------")
resultats = {}
for nom, (w1, w0) in modeles.items():
    y_pred = w1 * experience + w0
    mse = np.mean((salaire - y_pred) ** 2)
    resultats[nom] = (w1, w0, mse)
    print(f"   {nom}    | {w1:5.2f} | {w0:5.2f} | {mse:.2f}")

# c) Graphique
fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(experience, salaire, c='#333', s=40,
           zorder=3, label='Donnees')

couleurs = {"A": "#e74c3c", "B": "#2ecc71", "C": "#3498db"}
x_line = np.linspace(0, 21, 100)
```

 CC BY-NC-SA 4.0

```python
for nom, (w1, w0, mse) in resultats.items():
    ax.plot(x_line, w1 * x_line + w0,
            color=couleurs[nom], lw=2,
            label=f'{nom}: y={w1}x+{w0} (MSE={mse:.1f})')

ax.set_xlabel("Experience (annees)")
ax.set_ylabel("Salaire (kEUR)")
ax.set_title("Comparaison de 3 modeles lineaires")
ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout(); plt.show()

# d) Conclusion
print("\nLe modele B a le MSE le plus faible.")
print("C'est normal : B est la droite des moindres")
print("carres, qui minimise le MSE par construction.")
```

Sortie attendue :

```
Modele |  w1   |  w0   |   MSE
-------|-------|-------|--------
   A   |  1.50 | 33.00 | 27.03
   B   |  2.07 | 31.27 | 20.46
   C   |  2.80 | 28.00 | 21.31
```

Le modèle **B** a le MSE le plus faible (20,46), car c'est précisément la droite des moindres carrés — celle qui, **par définition**, minimise le MSE.

Le modèle C a un MSE très proche (21,31) : une pente un peu trop forte compensée par un biais plus bas. Le modèle A (27,03) sous-estime l'impact de l'expérience (pente trop faible).

Conclusion générale

La droite des moindres carrés est **optimale** par construction : aucun autre jeu de poids (w_1, w_0) ne peut donner un MSE plus faible sur les mêmes données. C'est ce que nous démontrerons formellement au **Jour 2** en introduisant l'**optimisation** et la **descente de gradient**.

Variante : boucle automatique

On peut automatiser la recherche en testant des milliers de poids : c'est l'idée de la **descente de gradient** du Jour 2.

© 2025 Mohamed EL AFRIT — IPSSI • www.mohamedelafrit.com • CC BY-NC-SA 4.0